

Comparaison entre les approches classiques et les approches neuronales pour l'apprentissage par renforcement

Rapport de Stage :
Enzo Claeysen, Cristal SMAC, de Mai à Juillet 2025

Sommaire :

Chapitre 1 : Explications et introduction des algorithmes étudiés

1. L'apprentissage par renforcement
2. Q-Learning
3. DQN

Chapitre 2 : Codage des problèmes

1. Introduction des problèmes traités
2. Gymnasium
3. La distribution des récompenses
4. Les observations

Chapitre 3 : Évaluer les performances

Chapitre 4 : Jeux avec adversaires

Chapitre 5 : Comparaison effective

Chapitre 1 :

Explications et introduction des algorithmes étudiés

1 / L'apprentissage par renforcement

L'apprentissage par renforcement est l'une des trois principales méthodes d'apprentissage automatique.

Son principe repose sur l'introduction d'un agent autonome au sein d'un environnement.

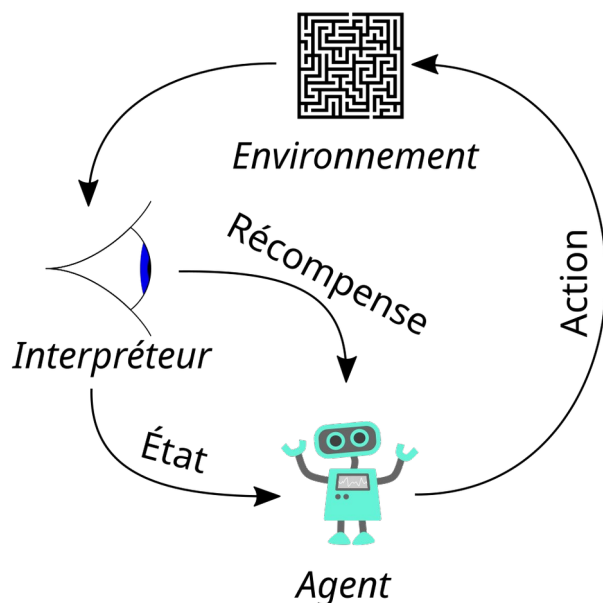
Un agent autonome correspond à tout ce qui peut observer et agir selon ce qu'il observe.

L'environnement correspond au cadre dans lequel évolue cet agent.

L'environnement admet un état qui peut être modifié.

En agissant, l'agent modifie l'état de l'environnement, l'environnement renvoie à l'agent sa prochaine observation ainsi que la récompense immédiate obtenue.

L'agent reçoit une succession de récompenses selon la suite d'actions qu'il effectue, l'objectif de l'agent est de maximiser la somme des récompenses obtenues au cours du temps.



Wikipédia / FR
Apprentissage par renforcement

2 / Le Q-Learning

Le Q-Learning tabulaire est un algorithme d'apprentissage par renforcement créé par Chris Watkins en 1989.

Son principe est d'estimer le cumul des récompenses qu'obtiendra l'agent s'il effectue l'action a sous l'état s de l'environnement.

Pour ce faire, le Q-Learning est conçu de façon à construire la fonction "Qualité" (fonction Q) qui a une paire (état, action) associe l'espérance de la somme pondérée des récompenses qu'obtiendra l'agent à partir de l'application de cette action dans cet état.

Cette méthode est dite "tabulaire" car la fonction Q est représentée par une table.

À chaque fois que l'agent reçoit une récompense, on met à jour la cellule (état, action) de la table qui correspond à l'aide de l'équation de Bellman

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Où

$R(s, a)$ correspond à la récompense renvoyée par l'environnement en effectuant l'action a dans l'état s .

$Q(s, a)$: La valeur de la fonction Q pour la paire (état s , action a)

alpha : Le taux d'apprentissage, qui détermine à quel point une nouvelle information est importante par rapport à ce qui est déjà appris.

gamma : facteur d'actualisation, qui détermine l'importance des récompenses futures

alpha et gamma sont donc des **paramètres** à définir avant le début de l'entraînement, ils ont une très grande importance.

Attention !

Afin d'éviter de converger vers une solution suboptimale, il faut que notre agent **explore**.

Pour cela, on fait en sorte que notre agent n'agit pas de la même façon selon s'il s'entraîne ou non.

S'il ne s'entraîne pas alors si l'agent observe l'état s , alors **il effectue l'action a qui maximise $Q(s, a)$**

Cette façon de sélectionner l'action à effectuer selon ce que l'on observe et selon la fonction Q s'appelle **la greedy-policy**

Lorsque **l'agent s'entraîne**, afin de découvrir de nouvelles stratégies, il utilisera **la epsilon-greedy policy**.

C'est-à-dire que l'agent aura une **probabilité epsilon d'effectuer une action aléatoire** et une probabilité $1 - \text{epsilon}$ d'agir selon la greedy-policy.

Le Q-Learning utilise 2 policy distinctes selon si l'agent s'entraîne ou non, on dit que c'est une **méthode off-policy**

epsilon est donc un nouveau paramètre introduit par ce problème :

si l'agent **n'explore pas suffisamment** lors de son entraînement, alors il peut ne jamais trouver de solution ou **ne trouver qu'une solution sub-optimale**

au contraire,

si l'agent **explore trop** alors il **perdra beaucoup de temps** lors de son entraînement à effectuer des actions qui ne lui permettent pas d'apprendre quoi que ce soit d'intéressant.

3 / Deep Q Network (DQN)

Et si la table Q devient trop grande ?
Cela arrive lorsque trop d'états et d'actions sont possibles.

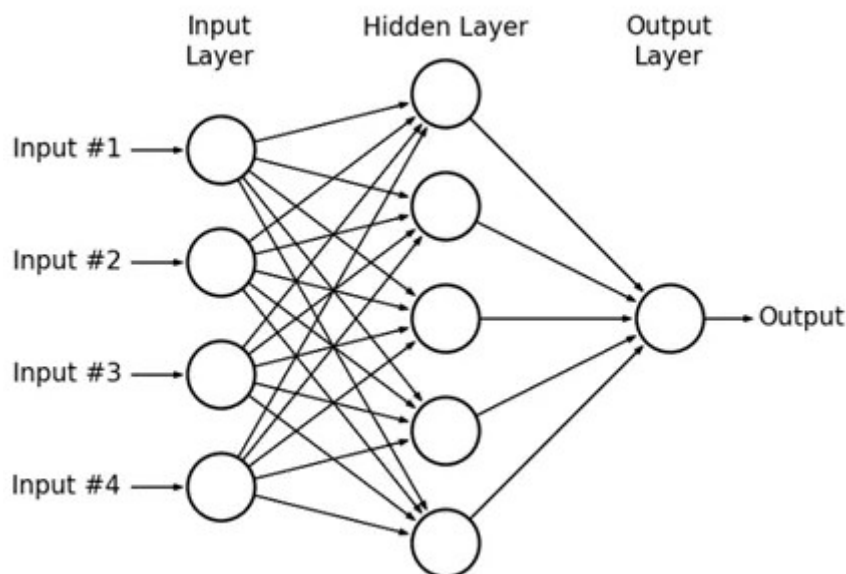
En effet, le nombre de "cases" dans la table est de Nombre d'états x nombre d'actions possibles
la taille de la table peut rapidement grandir.

Lorsque la table est trop grande, l'entraînement ralentit.
De plus, il se peut que la table soit trop grande pour la machine sur laquelle on utilise l'agent !

Il faut donc trouver une nouvelle façon de représenter la fonction Q.

On utilise un réseau de neurones artificiel.
DQN est donc un algorithme qui se base sur le même principe que le Q-Learning, la seule différence est le remplacement de la table par un réseau.

Voici comment on se représente un réseau "dense" (ceux que nous utilisons) :



Chaque sommet représente un neurone, chacun d'entre eux possède un biais.
Chaque arête est pondérée.

On utilise un "One Hot Encoding" ce qui signifie que l'ensemble des noeuds d'entrée renvoie 0 sauf l'un d'entre eux qui renvoie 1.

Un noeud reçoit la somme des sorties des noeuds de la couche précédente multiplié par les poids de l'arête reliant les deux sommets.

À cette somme on applique une fonction : la fonction d'activation. Ici on utilise la fonction ReLU : $x \Rightarrow \max(x, 0)$

Grâce à la rétropropagation du gradient, nous pouvons changer les poids et les biais afin que le réseau s'approche renvoie des estimations les plus proches possibles des observations.

Un avantage du réseau est qu'il est capable de modifier la valeur de Q pour plusieurs paires (action, état) par observation.

Néanmoins, ce phénomène est aussi sa faiblesse puisqu'il est bien plus instable.

Afin de mieux gérer cette instabilité, on utilise deux réseaux, le second réseau est appelé réseau cible.

Le réseau cible sera celui utilisé lors de la mise à jour du premier réseau.

Après un certain nombre de modifications du premier réseau, on le copie et sa copie devient le nouveau réseau cible.

Chapitre 2 : Codage des problèmes

1/Introduction des problèmes traités

1.1 : Couloir

Le premier problème consiste en une succession de cases.
Ces cases forment un couloir.

L'agent commence tout à gauche et doit arriver sur la case la plus à droite.

L'agent n'observe que sa position.

Deux actions sont possibles :

- Aller à gauche
- Aller à droite

Voici en exemple un couloir de taille 5

```
-----  
|D| | |F|      D pour début, F pour fin  
-----
```

1.2 : Labyrinthe

L'objectif est d'apprendre à un agent à résoudre un labyrinthe spécifique. Environnement basé sur "FrozenLake" de Gymnasium

Voici un exemple de labyrinthe :

SFFF S : Point de départ

FHFF F : case libre

FFHG H : Trou

G : L'objectif

Lorsque l'agent tombe dans un trou, l'épisode prend fin.
Il faut que l'agent aille du départ jusqu'à l'objectif sans tomber.

Nous pouvons générer une carte aléatoire qui possède au moins un chemin de S à G.

L'entraînement ne se fait donc que sur une unique carte puisque l'objectif est de résoudre une carte précisément.

1.3 : Livraison

On reprend le problème du labyrinthe, sauf que cette fois-ci on place une boîte B.

L'agent doit d'abord saisir la boîte, puis se diriger vers l'objectif.

Exemple :

SFFB
FFHH
HFFF
HHFG

On remarque que l'agent doit parfois rebrousser chemin une fois avoir atteint la boîte, c'est pourquoi il est nécessaire pour l'agent de savoir s'il a déjà pris la boîte ou non.

Pour le labyrinthe et la livraison, 4 actions sont possibles pour avancer dans chacune des 4 directions.

1.4 : Rome Alésia

Ce jeu oppose deux joueurs.

Un joueur joue Rome, l'autre joue Alésia.

L'objectif est de capturer la ville adverse.

Pour cela, il faut faire avancer le front sur la position de la ville adverse.

En début de partie, un certain nombre d'unités est attribué aux deux joueurs.

De plus, un certain nombre d'étapes est choisi pour séparer les villes du point de départ.

Le plateau se présente ainsi (pour 2 étapes):

R--|--|--M--|--|--A

R : Rome

A : Alésia

| : Une étape

M : Point de départ

Les deux joueurs choisissent un certain nombre d'unités simultanément.

Celui ayant le plus grand nombre fait avancer le front d'une étape vers la ville adverse.

Les deux joueurs perdent le montant donné d'unité.

Exemple : Si les deux joueurs commencent avec 50 unités.

Actions :

Rome : 10 Unités

Alésia : 20 Unités

Alésia utilise plus d'unité, le front avance vers Rome

R--|--X--|--|--A

Unités restantes :

40 pour Rome

30 pour Alésia

On recommence jusqu'à la fin de la partie.

Il y a match nul si les deux joueurs ne possèdent plus d'unités et que le front n'atteint aucune ville.

2/Gymnasium

Afin d'utiliser des bibliothèques telles que stable baselines, il est nécessaire de créer l'environnement en se basant sur Gymnasium

Gymnasium est une bibliothèque qui propose certains environnements, mais il est possible de créer ses propres environnements qui doivent suivre un "schéma" précis :

- Les environnements sont des classes qui héritent de `gymnasium.Env`
- Ils possèdent une méthode `__init__()` dans laquelle il faut définir les attributs `observation_space` et `action_space`
- Une méthode `step()` qui prend une action, l'applique à l'environnement et renvoie le nouvel état et la récompense
- Une méthode `reset()` qui prend une seed (pour les environnements stochastiques), réinitialise l'environnement et renvoie l'état initial.

3/La distribution des récompenses

La distribution des récompenses correspond à la façon dont l'environnement distribue les récompenses.

Nous avons constaté qu'une mauvaise distribution peut fortement impacter les performances des divers algorithmes d'apprentissage.

Par exemple, si dans le problème du labyrinthe, on ne donne pas de récompense négative à l'agent dans le cas où il tombe, alors les performances de l'apprentissage sont fortement dégradées.

Si on souhaite que l'agent trouve le chemin optimal, il faut lui attribuer une récompense négative plus modérée à chaque pas sinon il trouvera un chemin mais pas nécessairement le plus court.

Il faut aussi faire attention à ce que les récompenses négatives attribuées à chaque pas ne soient pas trop élevées sinon l'agent apprendra à courir vers le trou le plus proche plutôt que de trouver la sortie.

Une distribution des récompenses choisie correctement est donc essentielle puisque les récompenses dictent les objectifs de l'agent.

4/Les observations

Une autre partie à ne pas négliger est ce que l'agent peut observer.

Une bonne façon de savoir si les observations suffisent pour que l'agent puisse résoudre le problème est de se mettre à sa place.

Par exemple dans le problème du labyrinthe, si nous donnons à l'agent uniquement sa position comme information alors il ne peut qu'apprendre par essai-erreur en retenant "par coeur" le chemin à suivre.

Il n'y a donc aucun moyen pour l'agent de généraliser sa capacité à résoudre ce labyrinthe à d'autres labyrinthes puisqu'il ne possède pas suffisamment d'informations.

Au-delà de la sémantique des informations envoyées, on peut aussi changer la façon dont les observations sont codées.

Pour le Qlearning il n'y a pas vraiment de possibilité on peut certes changer les dimensions de la table mais le nombre de cases qu'elle contiendra sera strictement le même.

Néanmoins, pour DQN, des changements peuvent être observés.

Même en utilisant le One Hot Encoding pour la couche d'entrée, de multiples possibilités s'offrent à nous.

Pour le problème de la livraison, il est possible soit de coder la position et la possession de la boîte en un unique entier, auquel cas pour un labyrinthe comportant 16 cases, il y aura 32 noeuds pour la première couche.

Si on code ces deux informations séparément, nous aurions 16 noeuds d'entrée pour la position et 2 noeuds pour la possession de la boîte.

Lors des essais on remarque des résultats similaires en temps de convergence en étape (step), mais le plus faible nombre de noeuds dans le second cas amène à un temps en secondes plus court !

Expérimenter avec différentes manières de coder l'information (les observations / le retour du réseau) peut permettre de trouver une façon plus rapide d'obtenir le résultat voulu.